

VERIQO -- CORE TRUST PIPELINE TO ACTION CLOSURE

Response to Masih_terlalu_banyak_gap.docx, the reviewer's follow-up after the prior round's VERIQO_OS_TRUST_INTEGRATION_CLOSURE.md. That review named a "ROUND NEXT -- P1/P0 INTEGRATION CLOSURE" sequence, items A through H, and explicitly instructed: do not jump to P2 (real-world qualification) before A-H are closed. This document reports on that exact sequence, in that exact order, closed this round -- and, per the reviewer's own explicit correction, never claims more than it proves.

TERMINOLOGY, STATED UP FRONT

The reviewer's single sharpest correction in the review: the prior round's "End-to-End" / "system-level" language was ambiguous with "the whole VERIQO OS." This document, and every claim in it, uses "Core Trust Pipeline E2E" -- never "VERIQO OS E2E." The Core Trust Pipeline is exactly:

Evidence -> Manifest -> Hypothesis -> Finding -> AuthorizedFinding
-> Decision -> ActionAuthorization -> Ledger

This round extends that chain by one link (Decision -> Action Authorization) but it remains, honestly, the CORE pipeline only. It still does not reach, and this document does not claim it reaches:

External API -> Authentication -> Authorization -> Workflow -> Evidence -> ...
External insurance actor -> Broker -> Insurer -> Claim -> Evidence
-> Decision -> Settlement

What changed this round on that front specifically: items D and E below give the Core Trust Pipeline its first two REAL, live entry points (an HTTP API route and a Workflow DAG) instead of remaining callable only from Go test code -- but both entry points still terminate inside the Core Trust Pipeline's own boundary, not inside the external systems named above. That distinction is the whole point of this section.

A. DECISION IMMUTABILITY AUDIT -- CLOSED

The reviewer's exact concern: "Apakah Decision benar-benar sealed secara package boundary?" (Is Decision truly sealed at the package boundary?), with the named failure mode d.Outcome = "APPROVED" after construction.

TestDecisionIsStructurallyImmutableAfterConstruction (pkg/insurance/decision/decision_test.go) proves this mechanically, not by assertion: reflect.TypeOf(Ddecision{}) is walked field by field, and the test fails if ANY field's PkgPath is empty (i.e. exported) -- the exact structural condition that would make d.Outcome = "APPROVED" compile. It further confirms *Decision and Decision expose the identical method set (no pointer-receiver method exists that could mutate a shared value), and that every method's receiver type is the value type, never a pointer. Decision was already built this way in the prior round; this round adds the mechanical proof that was previously only assumed.

B. DECISION PROVENANCE / MERKLE-ANCHORING CLARITY -- CLOSED

The reviewer's exact question: "Apa sebenarnya yang di-anchor ke Merkle tree? ... decision_id, atau canonical(Decision + provenance)?"

Answered concretely by

TestOSTrustTamperMutationSensitivity/internally_consistent_forged_replacement_ledger_still_fails_against_the_anchored_root

(test/integration/os_trust_tamper_test.go): a fully self-consistent FORGED replacement ledger (one an attacker built honestly from their own fabricated content, so Auditor.VerifyChain alone would pass it) still produces a Merkle root that diverges from the real, independently anchored checkpoint root R1. The test logs both roots for direct inspection. This proves the anchor is sensitive to the full semantic payload (canonical(Decision + provenance)), not a bare, swappable decision_id -- a real attack class (chain replacement) that a naive decision_id-only anchor would not catch.

Rationale provenance (the reviewer's P1 note: Decision's rationale should have deterministic lineage back through AuthorizedFinding -> Evidence -> Manifest -> Hypothesis -> Policy) was already structurally true before this round -- Decision.findingHash and Decision.authorizationHash are copied verbatim from the AuthorizedFinding that authorized it, never re-derived or caller-supplied -- and remains unchanged this round.

C. TAMPER MUTATION TESTS -- CLOSED

The reviewer's exact spec: Run A -> valid root R1 -> mutate one byte in evidence/manifest/finding/decision/ledger payload -> expect R2 != R1 AND verification FAILS.

test/integration/os_trust_tamper_test.go's

TestOSTrustTamperMutationSensitivity implements this exactly, as six subtests: a manifest ManifestHash bit-flip, a manifest SHA256 bit-flip, a Finding Hash bit-flip, a Finding CONTENT change with a stale hash (the cheapest, most literal "one byte changed" attack), a ledger payload bit-flip caught by Auditor.VerifyChain's hash recomputation, and the forged-replacement-ledger case from item B above. Decision itself has no exposed field to flip by design (see item A); its own tamper-sensitivity is proven via the one real, mutable surface it produces -- the ledger payload.

INSURANCE REAL-WORLD WIRING -- CLOSED (KERNEL-TO-FACADE WIRING, NOT THE FULL REAL-WORLD NETWORK)

The reviewer's item 16, flagged as "yang paling penting" (the most important not to lose): pkg/insurance/decision was only the decision kernel, disconnected from this repository's own, much larger, already-real insurance claim orchestration (pkg/insurance/api.Facade, ~37 domain packages built across many prior rounds -- policy, party, claim, coverage, quantum, recovery, salvage, network, reserve, dispute, regulatory, payment, credential, and more).

This round adds Facade.DecideClaim (pkg/insurance/api/api.go), wiring the Facade's existing, real 15-stage claim lifecycle to the Decision Trust Boundary via the identical gated chain: cre.BuildFinding -> cre.AuthorizeGrounded -> decision.MakeDecision -> decision.AppendToLedger.

It is gated on the case having reached DOSSIER_GENERATED (every real

analysis step the Facade drives has already run) and on a hypothesis the Facade's OWN AnalyzeCausation call actually analyzed -- a caller cannot decide a claim against a hypothesis this case's own causation analysis never evaluated. Four tests in pkg/insurance/api/decideclaim_test.go prove the legitimate path and three refusals: before DOSSIER_GENERATED, against an unauthorized hypothesis, and against ungrounded evidence.

Honest scope: this closes the WIRING gap the reviewer named. It does NOT build the full Real-World Insurance Network (Policy -> Insured -> Broker -> Underwriter -> Insurer -> Reinsurer -> P&I -> Claim -> Surveyor -> Loss Adjuster -> Coverage Assessment -> Liability -> Quantum -> Settlement -> Recovery/Subrogation) or the cross-domain Insurance <-> Maritime <-> Cargo <-> Trade <-> Finance <-> eBL <-> Payment integration -- those remain P2, per the reviewer's own sequencing, and are not claimed here.

D. API IMPLEMENTATION + ENFORCEMENT -- CLOSED (LIVE BOUNDARY, HONESTLY SCOPED)

The reviewer's required shape: API -> Identity -> Authorization -> Validation -> Trusted pipeline, with the explicit prohibition "Tidak boleh ada API -> Decision constructor secara langsung."

New route POST /insurance/decide (veriqo/gateway/rest/insurance_decide_route.go), composed behind this gateway's PRE-EXISTING, real security stack (security.JWTMiddleware for Identity, security.RBACMiddleware for Authorization -- the same middleware /lifecycle/run_unified already used) -- this round adds the route, not the auth infrastructure. The handler's own job is only Validation (JSON decode, real field checks via claimworkflow.BuildClaimDecisionPlan) and invoking the Trusted pipeline (item E below). It never constructs a decision.Decision, cre.AuthorizedFinding, or finding.Finding directly.

Five tests in veriqo/gateway/rest/insurance_decide_route_test.go prove this over a genuine net/http round trip: the legitimate path (200, a real ledgered Decision), no Bearer token (401, zero ledger records), a valid token with the wrong role (403, zero ledger records), ungrounded evidence from an authenticated+authorized caller (422, zero ledger records), and the route's own nil-safety (404 when InsuranceLedger is not configured, matching every pre-existing caller of NewServer).

Honest scope: this is the Core Trust Pipeline's first live HTTP entry point. It is not a full claim-management REST API (there is no persisted case store reachable over HTTP -- that would require building the case-store/session layer this repository does not yet have, a materially larger undertaking left as a named next step, not silently implied as done).

E. WORKFLOW IMPLEMENTATION + ENFORCEMENT -- CLOSED (LIVE PATH, HONESTLY SCOPED)

The reviewer's required shape: Workflow -> Policy -> Evidence -> Finding -> Authorization -> Decision, with the explicit prohibition "Tidak boleh ada Workflow -> Decision constructor secara langsung."

New package pkg/insurance/claimworkflow builds a real veriqo/pkg/workflow.Plan -- this repository's own pre-existing Multi-Agent Runtime / Distributed Workflow engine (Planner, Scheduler,

Executor, checkpointing) -- with five steps: `finalize_evidence`, `build_hypothesis`, `build_finding`, `authorize_finding`, `decide`. Each step's Agent function only ever sees the named outputs of the steps it declared as dependencies (`workflow.AgentInput.Outputs`), which is what makes "the decide step reaches a Decision without a real AuthorizedFinding" structurally impossible, not merely conventional: a Plan whose decide step's `DependsOn` omits `authorize_finding` receives no such key in `Outputs` at all.

Three tests in `pkg/insurance/claimworkflow/claimworkflow_test.go` prove: the legitimate five-step run producing a real, ledgered Decision; a hand-crafted malicious Plan (built the way a real attacker would -- directly against `veriqo/pkg/workflow`, stripping decide's dependency on `authorize_finding`) refused with zero ledger records; and ungrounded evidence refused at `authorize_finding`, before decide ever runs. POST `/insurance/decide` (item D) uses this exact DAG as its own Trusted pipeline -- the two boundaries are not separate implementations of the same logic, they are the same implementation reached two different ways.

F. KNOWLEDGE/INTELLIGENCE BOUNDARY EXECUTABLE TESTS -- CLOSED

The reviewer accepted the prior round's "re-confirmed already closed" claim provisionally, but required proof the flow is Intelligence -> Hypothesis, never Intelligence -> Authoritative Finding directly: "salah satu prinsip terpenting VERIQO."

`test/integration/os_trust_intelligence_boundary_test.go`'s `TestIntelligenceOutputCanOnlyInformAHypothesisNeverAssertAFinding` drives a real, lifecycle-gated `inference.Recorder` through a genuine AI recommendation, then proves three shortcut attempts each fail: (1) injecting the AI's own recommended verdict directly onto a Hypothesis via `Add` -- discarded, `HypothesisSet.Add` always stores `StatusUnproven` regardless of caller input; (2) escalating a real Finding's `ConfidenceBasis` to the AI's recommended verdict rather than the hypothesis's real, evidence-derived status -- refused by `cre.Authorize`'s own verification; (3) citing a fabricated inference trace ID -- refused with `cre.ErrInferenceTraceNotFound`. The one legitimate path is then shown: a Finding's `ConfidenceBasis` honestly reflects the REAL derived status (`StatusContradicted`, since the AI's own "well-supported" recommendation was wrong and real evidence overrides it) and authorizes successfully.

G. DECISION -> ACTION AUTHORIZATION BOUNDARY -- CLOSED (DESIGN + IMPLEMENTATION)

The reviewer's new item 17, flagged as likely the next-generation P1/P0 boundary: after a Decision, what governs it being used for action (Approve claim -> Settlement; Insurance notification; Trade-finance action)? Required fields: decision authority, actor, policy, scope, expiry, permitted action, conditions, audit trail. Explicit prohibition: "Jangan sampai: trusted Decision -> arbitrary action."

New package `pkg/insurance/action` defines `ActionAuthorization` -- the identical sealed-type discipline as `AuthorizedFinding` and `Decision` (zero exported fields, value-receiver-only methods, proven via the same reflect-based mechanical test as item A). `AuthorizeAction` is the ONLY exported constructor, requiring a real, non-zero Decision, a non-empty actor/policyRef/scope, a known Action from a closed

four-value vocabulary (APPROVE_SETTLEMENT, SEND_NOTIFICATION, INITIATE_TRADE_FINANCE_ACTION, INITIATE_RECOVERY_ACTION -- the reviewer's own three named examples, generalised to categories, plus one more consistent with existing pkg/insurance/recovery work), and an expiry strictly after the authorization tick (an authorization already expired at the moment of minting is refused outright).

AuthorizeExecution is the one gate a downstream executor must call before actually taking an action: it independently re-verifies the authorization's own hash and its binding to the SPECIFIC Decision (VerifyActionAuthorization), then checks the attempting actor, attempted action, and attempted scope each exactly match what was granted, AND that the current tick has not passed the grant's expiry. Nineteen tests in pkg/insurance/action/action_test.go prove the legitimate path and every named bypass: an unauthoritative (zero) Decision, empty actor/policy/scope, an unknown action, an already-expired grant, a mismatched Decision, an expired authorization at execution time, and -- the sharpest three -- a different actor, a different action, and a different scope than the one actually granted, each independently refused.

test/integration/os_trust_action_boundary_test.go extends this to a full system-level proof: the real Core Trust Pipeline (Evidence -> ... -> Decision) continues one step further into Decision -> ActionAuthorization -> AuthorizeExecution, on the SAME real ledger the Decision itself was anchored to. Four bypass attempts (wrong actor, wrong action, wrong scope, expired grant) are each proven to leave ZERO new ledger entries; only the one legitimate execution produces the final ACTION_EXECUTED record. The finished ledger is DECISION_RECORDED -> ACTION_AUTHORIZATION_RECORDED -> ACTION_EXECUTED, independently hash-chain-verified.

Honest scope: this closes the reviewer's explicit ask -- "design dan implement authorization untuk tindakan downstream" -- as a real, tested, sealed boundary. It does not implement the downstream actions themselves (an actual settlement transfer, an actual notification dispatch, an actual trade-finance system call) -- AuthorizeExecution answers "is this specific actor, right now, permitted to take this specific action," and deliberately has no opinion on HOW that action is then carried out. Wiring AuthorizeExecution in front of a real settlement/notification/trade-finance integration is P2 work, named but not started, consistent with every other real-external-system item in this document.

H. MLETR PRIMARY-SOURCE VERIFICATION -- RETRIED, STILL NOT ACHIEVABLE THIS ROUND (HONESTLY DOCUMENTED)

Retried against two additional domains this round (academy.iccwbo.org, handwiki.org), both EGRESS_BLOCKED -- six distinct domains now blocked across two rounds, reading as a blanket network-egress policy rather than a per-domain outage a further retry would likely fix. WebSearch (a separate, unblocked tool) did surface real, sourced synthesis for Articles 10, 11, 15, 17, and 18 this round -- documented in full, with source URLs, in a new "Primary-source verification retry (H)" section of VERIQO_MLETR_EBL_CONFORMANCE_MAPPING_V0_2.md. Critically, that synthesis appears to CONTRADICT the prior round's own article-number guesses (Article 12 was guessed as "change of medium"; this round's sources instead name Articles 17 and 18 as the change-of-medium pair)

-- flagged prominently in the document as a correction, downgrading the prior round's "HIGHER confidence" framing on Articles 9/11/12 rather than leaving the contradiction silently unresolved. Articles 13, 16, 17, and 18 remain explicitly LOW CONFIDENCE and are not marked complete, per the reviewer's own explicit instruction.

UPDATED STATUS TABLE

Legend: [GREEN] closed and tested this round or a prior round • [YELLOW] closed for a bounded scope, honestly flagged where it stops • [RED] not started, named as P2

Domain	Status
---	---
Evidence integrity	[GREEN]
Manifest integrity	[GREEN]
Hypothesis controls	[GREEN]
Finding controls	[GREEN]
AuthorizedFinding	[GREEN]
Insurance Decision constructor	[GREEN]
Unauthorized Decision prevention	[GREEN]
Decision -> Ledger	[GREEN]
Decision immutability (mechanically proven)	[GREEN] (A, this round)
Decision Merkle-anchoring clarity (canonical(Decision+provenance), not bare ID)	[GREEN] (B, this round)
Tamper mutation sensitivity (R1 vs R2 divergence, every artifact)	[GREEN] (C, this round)
Core Trust Pipeline chain (Evidence->...->Ledger)	[GREEN]
Core Trust Pipeline deterministic replay	[GREEN]
Merkle-root convergence	[GREEN]
Direct bypass tests (structural)	[GREEN]
Intelligence -> Hypothesis boundary (executable adversarial proof)	[GREEN] (F, this round)
Insurance Facade -> Decision Trust Boundary wiring	[GREEN] (this round)
API live enforcement (POST /insurance/decide, Identity+Authorization+Validation)	[GREEN] (D, this round)
Workflow live enforcement (claimworkflow DAG, bypass-tested)	[GREEN] (E, this round)
Decision -> Action Authorization Boundary	[GREEN] (G, this round)
OS-wide bypass resistance beyond these two entry points	[YELLOW] -- 2 of 6 named boundaries (API, Workflow) now have live orchestration to attack and have been attacked; Storage/Replay boundaries remain structural-only per the prior round's own honest scoping
MLETR primary verification	[YELLOW] -- retried, still not primary-source-verified; second-hand sourced synthesis added and reconciled against prior guesses (H, this round)
eBL real-world interoperability	[RED] P2
Insurance network integration (full real-world model)	[RED] P2
Real insurers/P&I/reinsurers	[RED] P2
Real counterparties	[RED] P2
Real data qualification	[RED] P2
Independent audit	[RED] P2
Downstream action execution (settlement/notification/trade-finance systems themselves)	[RED] P2 (the AUTHORIZATION boundary in front of them is [GREEN], per G)

VERIFICATION

gofmt -l .

clean

go build ./...	clean
go vet ./...	clean
go test -race (decision, action, claimworkflow, api, cre, causation, manifest, audit, workflow, gateway/rest, test/integration)	225 PASS, 0 FAIL, no data races
go test ./...	full repository suite, 0 FAIL
pkg/insurance/guardrails whole-tree scan	7/7 PASS, re-confirmed reaching every new package this round added

Every negative/authorization check added this round was verified via break-test-fix-restore: temporarily disabled, confirmed the exact expected diagnostic, restored. See the individual test files named in each section above for the specific assertions.

HONEST SCOPE BOUNDARY

- This document, and every "Core Trust Pipeline E2E" claim in it, covers Evidence through Decision through Action Authorization. It does not cover, and does not claim to cover, the full external API/Workflow/Insurance/Maritime/Trade-Finance/eBL chain the reviewer's own diagram named.
- The two new live entry points (API, Workflow) terminate inside the Core Trust Pipeline's own boundary. Neither is a persisted, production claim-management system; both are honestly scoped, additive routes/DAGs proving the boundary is live-attackable and resistant, not a claim that the full external system now exists.
- AuthorizeExecution (item G) is an authorization gate, not an action executor. No settlement transfer, notification dispatch, or trade-finance system call is implemented by this round's work.
- MLETR Articles 9-18 remain a technical-evidentiary self-assessment, not a legal opinion, and -- after two rounds and six distinct blocked domains -- still have not been checked against primary UNCITRAL source text by this session.
- This remains Level 2 (Engineering Verified) work on this engagement's own 4-level maturity model. Level 3 (Controlled External Validation) and Level 4 (Production Proven) require real external counterparties, real data, and independent audit -- P2, per the reviewer's own sequencing, and explicitly not started.